LANDrop

BÁO CÁO
PHÂN TÍCH BACKEND

Kiến trúc, nghiệp vụ, dữ liệu,
giao thức và mức độ hoàn thiện

258 tệp	182 Java production	66 Java test/helper
7 migration SQL	10 bảng nghiệp vụ	2 bộ server realtime

Ngày báo cáo: 05/10/2026
Repository: C:/TheAnhproject/LANDrop
Phạm vi: backend, đối chiếu tài liệu và cấu hình liên quan

Báo cáo phản ánh code đang có trong cây làm việc. Nhận xét phân tích tĩnh, kết quả test thực tế và các phần chưa xác minh được phân biệt trong nội dung.

Mục lục

1. Nhận định tổng quan 3

2. Phạm vi và phương pháp kiểm chứng 4

3. Kiến trúc tổng thể và ranh giới trách nhiệm 5

4. Công nghệ, build và các chương trình khởi chạy 6

5. Vòng đời server và cấu hình 7

6. Discovery và kết nối trực tiếp 8

7. Giao thức TCP và mã hóa P2P 9

8. Truyền file một peer, resume và file lớn 10

9. Tải file từ nhiều peer và chức năng LAN bổ sung 11

10. Tài khoản, phiên và thiết bị 12

11. Workspace, phân quyền và chat bền vững 13

12. Hai bộ WebSocket và sự khác biệt khi tích hợp 14

13. Metadata file/folder và transfer có DB 15

14. Mô hình dữ liệu và migration 16

15. Control Center và quan sát vận hành 17

16. Bản đồ API để tích hợp 18

17. Kiểm thử thực tế và độ tin cậy của bằng chứng 19

18. Các phát hiện và thứ tự ưu tiên 20

19. Đề xuất lộ trình hoàn thiện 21

20. Kết luận về dự án 22

Phụ lục A. Căn cứ mã nguồn chính 23

1. Nhận định tổng quan

LANDrop là một hệ thống chia sẻ dữ liệu và trao đổi thông tin giữa các thiết bị trong mạng LAN, được mở rộng thêm nền tảng cộng tác theo nhóm. Backend Java vừa đóng vai trò thư viện mạng chạy ở phía peer, vừa chứa các chương trình server trung tâm. Vì vậy, “backend” ở dự án này có phạm vi rộng hơn một REST API thông thường.

Tôi hiểu dự án qua ba trách nhiệm lớn: **tìm thiết bị và duy trì kết nối**, **truyền nội dung trực tiếp giữa các peer**, và **quản lý danh tính cùng dữ liệu cộng tác**. Phần truyền file có chia chunk, tiếp tục sau gián đoạn, retry, tải từ nhiều nguồn và kiểm tra SHA-256. Phần server có đăng nhập, phiên làm việc, đăng ký thiết bị, workspace, channel, lịch sử chat, metadata file và bảng quản trị.

Điểm mạnh nổi bật là chiều sâu của engine truyền file và sự phân tách giữa dữ liệu file với thông tin điều phối. Điểm cần hoàn thiện nhất là sự nhất quán giữa các đường khởi chạy server, các cơ chế xác thực, các hợp đồng realtime và giới hạn truyền file lớn. Mã nguồn có nhiều thành phần thực tế để phát triển tiếp; bằng chứng hiện có chưa đủ để kết luận hệ thống đã sẵn sàng vận hành production trọn vẹn.

Câu hỏi	Cách hiểu từ mã nguồn
Sản phẩm giải quyết việc gì?	Gửi file, chat và phối hợp theo nhóm trong môi trường nhiều thiết bị, lấy LAN làm nền tảng truyền dữ liệu trực tiếp.
File có đi qua PostgreSQL không?	Các luồng được đọc lưu metadata và trạng thái trong DB; byte file đi qua TCP giữa peer.
Có cần Internet cho mọi chức năng?	Engine LAN và UDP discovery có thể hoạt động độc lập; chức năng tài khoản/workspace cần server và PostgreSQL. Nếu PostgreSQL nằm trên cloud, phần đó phụ thuộc kết nối tới cloud.
Có phải microservices không?	Các package cùng nằm trong một Maven module/JAR; đây là kiến trúc nhiều thành phần trong một codebase, với nhiều entrypoint.
Prisma có thực thi truy vấn Java không?	Backend Java thực thi SQL bằng JDBC và HikariCP. Prisma schema là mô tả liên quan ngoài module Java.
Có lưu file cloud hoàn chỉnh không?	Có các tên storage type, nhưng chưa thấy implementation upload/download object storage trong các luồng backend được phân tích.

2. Phạm vi và phương pháp kiểm chứng

Danh mục backend có **258 tệp**, gồm **182 tệp Java production**, **66 tệp Java kiểm thử/hỗ trợ kiểm thử**, **7 migration SQL**, **1 trang HTML quản trị**, **1 POM** và **1 README**. Không tính `backend/target`, vì đây là đầu ra build và kiểm thử. Danh mục đi kèm liệt kê từng tệp, số dòng, kích thước và SHA-256 để xác định snapshot.

Phân tích kết hợp kiểm kê toàn bộ tệp, đọc cấu trúc lớp/phương thức/DTO, đọc sâu các luồng thực thi chính, đối chiếu migration và tài liệu, rồi chạy kiểm thử backend. Các kết luận chức năng dựa trên code và wiring thực tế. Tên test cho biết ý định kiểm thử; chỉ kết quả chạy mới cho biết test được thực thi thành công trong môi trường này.

Tài liệu ngoài backend được dùng để đối chiếu gồm `AGENTS.md`, POM gốc, `.env.example`, Prisma schema, tài liệu kiến trúc, phạm vi core, trạng thái mở rộng, các hợp đồng Foundation/Auth/Workspace/Chat/File và hướng dẫn kiểm thử. Các script khởi chạy cùng Dockerfile được kiểm tra ở những điểm ảnh hưởng đến backend.

Giới hạn của báo cáo: đây là phân tích mã nguồn và kiểm chứng tại máy hiện tại; chưa chạy trên hai máy LAN thật, chưa chạy PostgreSQL integration bằng database riêng, chưa thử file nhiều gigabyte và chưa triển khai container. Không đọc hoặc đưa giá trị bí mật từ `.env` vào báo cáo. Không thay đổi mã nghiệp vụ để giải quyết các nhận xét.

Trạng thái cây làm việc: POM gốc hiện chỉ khai báo module `backend`, cùng frontend web ngoài Maven. Mô tả “hai module Java Maven” trong hướng dẫn ban đầu cần được hiểu là bối cảnh trước đó. Trong lúc lập báo cáo, tài liệu và script demo được tổ chức lại, README backend được cập nhật, và thư mục Surefire có kết quả một lần chạy khác. Danh mục tệp được chốt khi xuất báo cáo; kết quả kiểm thử trong báo cáo và CSV được giữ theo log riêng của lần chạy kết thúc lúc 21:48:44 ngày 05/10/2026. Báo cáo không xác nhận mức độ hoàn thiện frontend mới.

3. Kiến trúc tổng thể và ranh giới trách nhiệm

Có hai lớp giao tiếp quan trọng. **Lớp quản lý và điều phối** dùng HTTP/WebSocket để xác thực, truy vấn dữ liệu, cập nhật presence, đăng ký nhận sự kiện và gửi yêu cầu truyền file. **Lớp truyền nội dung** dùng socket TCP của peer để chuyển chunk file, chat LAN và clipboard. Việc một yêu cầu transfer được chấp nhận trên server chưa có nghĩa file đã được gửi xong trên TCP.

```
                LỚP QUẢN LÝ VÀ ĐIỀU PHỐI
Client A ---- HTTP / WebSocket ---- Server ---- JDBC ---- PostgreSQL
Client B ---- HTTP / WebSocket -------|
                | tài khoản, thiết bị, nhóm, tin nhắn, metadata

                LỚP TRUYỀN NỘI DUNG LAN
Peer A ================ TCP trực tiếp ================ Peer B
        chat, clipboard, chunk file, trao đổi khóa
Peer C ================ TCP chunk ==================== Peer B
        nguồn bổ sung khi tải nhiều peer

Discovery: TCP registry 8888 hoặc UDP multicast 8889
```

Nhóm package	Vai trò	Thành phần tiêu biểu
`protocol`	Đóng/giải mã packet, loại message, correlation ID	Packet, PacketEncoder, PacketDecoder, MessageType
`peer`, `network`	Socket TCP, handshake, reader loop và CLI/demo	PeerConnection, PeerListener, PeerServer, Main
`discovery`	Tìm peer, heartbeat, danh bạ peer và multicast	DiscoveryServer, DiscoveryClient, PeerRegistry, UdpDiscoveryService
`transfer`	Engine gửi/nhận file, resume và tải nhiều nguồn	FileTransferService, MultiPeerDownloadCoordinator, ChunkManager
`security`	Trao đổi khóa và mã hóa nội dung P2P	P2pCryptoSession
`auth`, `device`	Tài khoản, JWT, phiên và thiết bị	AuthService, SessionVerifier, DeviceService
`workspace`, `database.service`	Workspace, thành viên và phân quyền	WorkspaceHttpServer, WorkspaceAccess, WorkspaceService
`chat.service`, `chat.server`	Chat nhóm được lưu DB và REST API	ChannelChatService, ChatHttpServer
`realtime`	Gateway chat/file theo bộ hợp đồng mở rộng	RealtimeServer, JdbcRealtimeStore, ConnectionRegistry
`server`	Server Foundation và điều phối kết nối runtime	ServerBootstrap, FoundationRealtimeServer, ClientRegistry
`file`	Metadata file/folder và transfer lưu DB	FileManagementService, TransferCoordinationService
`admin`	Control Center, quản trị và audit log	AdminHttpServer, AdminControlCenterService
`database`	Connection pool, SQL repository, migration và seed	DatabaseManager, DatabaseMigrator, các Repository
`config`, `util`	Cấu hình và logging dùng chung	AppConfig, AppLogger

Phân lớp service/repository tương đối rõ, nhưng có sự pha trộn: `ChannelChatService` viết SQL trực tiếp, `FileManagementService` tự truy vấn membership, trong khi workspace sử dụng repository có transaction. Điều này làm policy và hành vi giao dịch khó thống nhất khi chức năng mở rộng.

4. Công nghệ, build và các chương trình khởi chạy

Module backend là Maven JAR, phiên bản artifact `0.1.0-SNAPSHOT`, compiler release **21**. Backend không dùng Spring Boot hay ORM Java trong các dependency được khai báo; HTTP sử dụng `com.sun.net.httpserver.HttpServer`, WebSocket sử dụng Java-WebSocket, dữ liệu JSON dùng Jackson.

Thành phần	Phiên bản trong POM	Mục đích
Java-WebSocket	1.6.0	HTTP upgrade và WebSocket server
Jackson databind	2.17.0	Chuyển đổi JSON, request/response và discovery
PostgreSQL JDBC	42.7.3	Kết nối PostgreSQL
HikariCP	5.1.0	Quản lý pool kết nối DB
SLF4J API/simple	2.0.13	Logging của thư viện và DB
JUnit Jupiter	5.10.2	Kiểm thử
Maven Surefire	3.2.5	Chạy JUnit trong Maven

Đây là phiên bản được khai báo tại snapshot, không phải đánh giá các phiên bản mới nhất. Virtual thread được dùng cho HTTP executor, reader của peer và các worker truyền file; một số lớp demo cũ như `PeerServer` vẫn dùng cached thread pool thông thường. Không nên mô tả mọi thành phần là virtual thread.

Ba entrypoint có mục đích khác nhau:

- `com.landrop.server.LandropServerMain`: server Foundation đầy đủ, qua `ServerBootstrap`, HTTP 3000 và WebSocket 3001 mặc định; có Control Center và discovery nhúng ngoài môi trường test.
- `com.landrop.auth.server.AuthHttpServer`: đường chạy Auth/Realtime trước đó, được `scripts/demo/start-auth.bat` gọi; có workspace/chat/file nhưng wiring quản trị, phiên và thiết bị khác Foundation.
- `com.landrop.network.Main`: CLI với các chế độ peer/discovery/server/client và đường demo phase 2. Manifest của JAR vẫn trỏ vào lớp này.

`DiscoveryServer`, `DatabaseCli`, `RealtimeClient` và `AdminPasswordResetMain` là những chương trình hỗ trợ khác. `AdminPasswordResetMain` hiện ném `UnsupportedOperationException`, nên không thể coi nó là công cụ reset mật khẩu hoạt động.

Lưu ý vận hành: `network.Main` chế độ `server` là server chat/demo, không đồng nghĩa `LandropServerMain`. JAR tạo bởi Maven JAR plugin không đóng gói toàn bộ dependency thành fat JAR. Dockerfile dùng classpath `landrop.jar` và `lib/*` để cấp dependency.

5. Vòng đời server và cấu hình

`ServerBootstrap.start()` thực hiện kiểm tra mật khẩu admin nếu environment là production, kiểm tra kết nối DB, khởi tạo các service có DB khi kết nối được, tạo bảng quản trị, mở HTTP, mở WebSocket và chờ listener sẵn sàng. Sau đó nó thử mở discovery TCP trên **8888** nếu không phải môi trường test, đăng ký shutdown hook và in thông tin các địa chỉ LAN.

Shutdown đóng các kết nối realtime, xóa registry, dừng HTTP/discovery rồi đóng pool DB. Đây là thiết kế hữu ích để thu hồi tài nguyên. Một điểm cần xem thêm là rollback tài nguyên khi startup hỏng giữa chừng: HTTP được mở trước WebSocket, nhưng chưa có một khối cleanup startup tương đương shutdown khi mọi bước chưa hoàn tất.

Khi thiếu DB, Foundation vẫn có thể mở HTTP health/status và phần điều phối in-memory. Các route auth/workspace/chat/file được gắn có điều kiện khi service DB được tạo. Vì vậy, “server vẫn chạy” không có nghĩa “mọi API vẫn dùng được”. `/health` trả `ok` và uptime, không thực hiện kiểm tra readiness của toàn bộ hệ thống dữ liệu.

Cấu hình	Mặc định / hành vi
HTTP/WebSocket Foundation	`0.0.0.0:3000` và port `3001`
Heartbeat Foundation	Interval 15 giây, timeout 45 giây
Discovery TCP	Port 8888; cấu hình riêng ở AppConfig, nhưng bootstrap nhúng hiện hardcode 8888
Discovery UDP	Nhóm 239.255.88.88, port 8889, beacon mỗi giây, timeout peer 10 giây
Peer TCP	AppConfig mặc định 5001, advertised host 127.0.0.1
Transfer	Chunk mặc định 1 MiB, timeout chunk 5 giây, retry tối đa 3 lần
Reconnect single peer	Tối đa 5 lần, delay 1/2/4/8/10 giây
DB pool	Max 10, min idle 2, connection timeout 30 giây
JWT	Access 15 phút, refresh 7 ngày

`AppConfig` ưu tiên JVM system property, sau đó environment rồi mặc định. `AuthConfig` và `DatabaseConfig` còn đọc `.env` ở một số thư mục tương đối. `ServerConfig.load()` không tự dùng cùng cơ chế đọc `.env`; launcher Node có logic nạp `.env`/`.env.local` riêng. Do đó phải xác định rõ chương trình nào đang nạp cấu hình.

`ServerConfigManager` lưu `data/server-config.json`, phân biệt current/pending và báo `requiresRestart` khi đổi host/port. Tuy nhiên, `ServerConfig.load()` không nạp file đó làm cấu hình hiệu lực khi restart; các thay đổi heartbeat/log cũng chưa được nối trực tiếp tới instance đang chạy. Đây là chức năng quản lý cấu hình đã có giao diện và lưu file, nhưng đường áp dụng chưa hoàn chỉnh.

6. Discovery và kết nối trực tiếp

6.1. Discovery tập trung qua TCP

Peer gửi thông tin ID, tên và endpoint vào `DiscoveryServer`. `PeerRegistry` lưu bằng concurrent map, kiểm tra peer hợp lệ, từ chối đăng ký trùng ID đang active, cập nhật last-seen khi có heartbeat và loại peer quá hạn. Peer hỏi danh sách thường loại chính mình rồi chọn một peer để kết nối TCP trực tiếp.

Các message chính là `REGISTER`, `REGISTER_ACK`, `PEER_LIST_REQUEST`, `PEER_LIST`, `HEARTBEAT`, `HEARTBEAT_ACK`, `UNREGISTER`. Discovery không chuyển tiếp byte file. TCP registry cũng chưa phải hệ thống danh tính tài khoản: peer ID và tên quảng bá không được xác minh bằng user/session trong luồng này.

6.2. Discovery phân tán qua UDP multicast

`UdpDiscoveryService` quảng bá presence vào nhóm multicast, thử tham gia qua các network interface, bỏ qua chính mình, kiểm tra độ dài ID/tên và port. IP peer được lấy từ địa chỉ nguồn datagram, thay cho giá trị host tự khai báo. Peer im lặng quá 10 giây bị loại khỏi danh sách.

UDP discovery giúp chế độ LAN tìm peer mà không cần TCP discovery server. Việc tìm thấy peer không chứng minh peer đáng tin; multicast hiện chưa có chữ ký hoặc binding với danh tính tài khoản. Khả năng hoạt động trên router, Wi-Fi/AP isolation, VPN hoặc interface cụ thể cần được kiểm tra ở mạng thật.

6.3. Kết nối peer

`PeerConnection` quản lý socket, input/output stream, trạng thái kết nối, reader virtual thread, handler theo message type và ping/pong bất đồng bộ. Gửi packet có write lock để tránh xen kẽ dữ liệu từ nhiều thread. HELLO trao đổi tên và purpose `CHAT`/`FILE_TRANSFER`; `awaitReady()` và `awaitSecure()` là hai mốc khác nhau: handshake xong chưa đồng nghĩa phiên mã hóa đã sẵn sàng.

Purpose cho phép phía tích hợp dùng kết nối file riêng để giữ chat đáp ứng trong lúc tải. Đây là thông tin handshake và cấu trúc tổ chức kết nối, không phải một cơ chế ưu tiên băng thông ở mức hệ điều hành.

7. Giao thức TCP và mã hóa P2P

7.1. Packet có header 14 byte

Trường	Kích thước	Ý nghĩa
Version	1 byte	Protocol version hiện tại = 1
Message type	1 byte	Chọn loại handler
Payload length	4 byte	Độ dài payload
Request ID	8 byte	Correlation giữa request/response
Payload	Theo length	JSON, văn bản UTF-8 hoặc binary

Encoder dùng `DataOutputStream`; decoder đọc đủ từng trường và payload, kiểm tra version, type, độ dài âm, payload vượt giới hạn và dữ liệu bị cắt. `Packet` sao chép byte array để hạn chế thay đổi từ bên ngoài. `RequestIdGenerator` dùng AtomicLong.

MessageType hiện có **25 loại**, bao phủ chat/ping, discovery, offer/request/chunk/ACK, resume, file query, clipboard, encrypted payload và key exchange. Giới hạn mặc định của decoder là **2 MiB cho payload**, không phải kích thước tối đa của cả file.

Encoder/decoder xác nhận header dài 14 byte. README được đọc ở đầu phiên từng ghi “8-Byte Header”; bản README cập nhật trong lúc lập báo cáo đã bỏ mô tả đó. Luồng single-peer hiện chủ yếu là receiver chủ động `CHUNK_REQUEST` rồi nhận `CHUNK_RESPONSE`; `FILE_DATA` vẫn là đường tương thích cũ.

7.2. Phiên mã hóa

`P2pCryptoSession` sinh keypair EC curve `secp256r1`, trao đổi public key bằng ECDH, dùng SHA-256 của shared secret làm khóa AES 256 bit. Nội dung packet được mã hóa bằng AES/GCM, IV ngẫu nhiên 12 byte và authentication tag 128 bit. Packet bên trong được serialize rồi bọc trong `ENCRYPTED_PAYLOAD`. Có fingerprint 6 chữ số để ứng dụng hiển thị.

Test hiện có kiểm tra hai phía tính cùng khóa, mã hóa hai chiều và từ chối ciphertext bị sửa. Cần phân biệt ba thuộc tính: SHA-256 xác nhận toàn vẹn file so với hash kỳ vọng; AES-GCM bảo vệ nội dung trong phiên dùng đúng khóa; xác thực peer cần cơ chế riêng để biết khóa đó thực sự thuộc ai.

Code chưa cho thấy public key P2P được chứng thực hoặc được ràng buộc với user/device server. Fingerprint được tạo nhưng chưa có bước engine bắt buộc người dùng xác nhận. Mặc định `ENFORCE_E2E_ENCRYPTION=false`, nên gửi trước khi key agreement hoàn tất có thể đi plaintext. Flag enforcement hiện kiểm tra đường gửi; dispatcher vẫn xử lý packet ứng dụng plaintext ở đường nhận. Vì vậy không nên mô tả phiên hiện tại là xác thực peer hai chiều hoàn chỉnh.

8. Truyền file một peer, resume và file lớn

8.1. Luồng truyền thành công

1. Sender đọc thông tin file, tính SHA-256, chọn chunk size, tạo transfer ID và gửi `FILE_OFFER`.
2. Receiver kiểm tra metadata và thông báo cho phía tích hợp chấp nhận hoặc từ chối.
3. Khi chấp nhận, receiver chọn tên đích tránh trùng, mở `.landrop.part` và bitmap các chunk đã nhận.
4. Receiver lần lượt yêu cầu chunk còn thiếu. Sender đọc phần tương ứng bằng FileChannel rồi gửi binary response.
5. Receiver kiểm tra index/length, ghi vào đúng offset, cập nhật progress và lưu `.landrop-meta`.
6. Khi đủ chunk, receiver kiểm tra kích thước file và SHA-256, đổi file tạm thành file đích rồi gửi ACK hoàn tất.

Hash được tính theo luồng với buffer 64 KiB; file không phải được nạp nguyên vào một byte array tại engine. Đây là cơ sở tốt để xử lý file lớn. Tuy vậy, còn chi phí đọc toàn file trước khi offer, kiểm tra hash khi resume và đọc lại toàn file sau khi nhận.

`FileTransferStatus` có PENDING, ACCEPTED, TRANSFERRING, INTERRUPTED, RECONNECTING, RESUMING, VERIFYING và các trạng thái kết thúc. Engine còn hỗ trợ reject, cancel và pause. Progress phía nhận được tiết lưu khoảng 100 ms để giảm lượng callback.

8.2. Resume

Bitmap đánh dấu chunk hoàn thành được lưu vào `.landrop-meta`, có magic/version và ghi qua file tạm trước khi đổi tên. Khi mất kết nối, receiver lưu trạng thái, đóng channel, chuyển transfer sang INTERRUPTED và có thể reconnect theo backoff. `RESUME_REQUEST` mang metadata và bitmap; sender kiểm tra kích thước, chunk size, số chunk, hash và việc file nguồn thay đổi trước khi chấp nhận.

Điều này cung cấp resume cho phiên transfer mà hai phía còn thông tin. Chưa có catalog bền vững đầy đủ chứa transfer ID, endpoint, tên đích, hash và sender context để tự khôi phục mọi transfer sau khi hai ứng dụng đều khởi động lại. `FileTransferService.close()` cũng có đường thất bại và xóa partial đang active. Vì vậy không nên quảng bá resume “sau mọi loại restart” chỉ dựa trên sự tồn tại của bitmap.

8.3. Bất nhất với chunk 2 MiB

`FileChunk.determineOptimalChunkSize()` chọn **2 MiB** khi file lớn hơn **2 GiB**, còn `PacketDecoder` giới hạn payload đúng **2 MiB**. Một CHUNK_RESPONSE chứa thêm 1 byte độ dài ID, ID, index và length. Với transfer ID UUID 36 byte, payload cho chunk 2 MiB là **2.097.197 byte**, vượt **2.097.152 byte** trước cả khi mã hóa. Khi mã hóa, inner header và IV/tag còn làm kích thước tăng thêm.

Đây là bất nhất xác định từ cách tính trong code, không phải kết quả một lần truyền file 3 GiB đã chạy. Cần thống nhất công thức max chunk theo max payload trừ overhead hoặc tăng giới hạn packet một cách có kiểm soát, rồi thêm kiểm thử ở ranh giới và file >2 GiB.

8.4. Những giới hạn khác cần hiểu đúng

`FileOfferMessage` cho phép hash trống để tương thích, và receiver bỏ bước so sánh hash kỳ vọng nếu trống. Vì vậy, bảo đảm “hash nguồn bằng hash đích” chỉ đúng khi offer cung cấp hash hợp lệ. Đồng thời, constructor chuẩn hóa file size âm về 0 khiến validation sau đó không còn nhìn thấy số âm gốc.

Các helper `transferTo`/`transferFrom` có tồn tại, nhưng đường chunk đang đọc vào buffer và serialize packet; sự tồn tại helper chưa chứng minh đường truyền chính đã dùng zero-copy. Ngoài ra, nhiều getter cấu hình retry/reconnect trong AppConfig không được constructor engine tự áp dụng; engine vẫn có constant và setter riêng.

9. Tải file từ nhiều peer và chức năng LAN bổ sung

`MultiPeerDownloadCoordinator` hỏi các peer có cùng file bằng hash/size, tiếp nhận nguồn phù hợp, tạo worker cho mỗi nguồn và mở file tạm chung. `ChunkManager` giữ trạng thái MISSING, REQUESTED, TRANSFERRING, COMPLETED. `ChunkScheduler` phân việc round-robin, mặc định giới hạn 2 chunk đã giao cho một peer. Mỗi worker hiện xử lý queue theo tuần tự; hai chunk active không đồng nghĩa hai request đồng thời trên cùng worker.

Các worker ghi vào vị trí riêng bằng `FileChannel.write(buffer, offset)`, nên có thể lắp file mà không phải nối từng phần theo thứ tự nhận. Khi peer mất kết nối, chunk chưa xong được trả về MISSING để nguồn còn lại tải. Peer mới có thể tham gia khi tải đang chạy. Khi mọi chunk hoàn tất, coordinator kiểm tra kích thước và hash rồi chuyển file sang đích.

Đây là tải nhiều nguồn của **cùng một bản file**, không phải đồng bộ file phân tán toàn diện. Không thấy cơ chế định tuyến Internet/NAT traversal, tracker phân tán hoặc object storage trong coordinator. Nếu tất cả nguồn mất kết nối, phiên tải thất bại. Đường hoàn tất multi-peer hiện dùng ATOMIC_MOVE mà không có fallback như single-peer, nên hành vi giữa filesystem cần được kiểm tra.

`SharedFileScanner` quét thư mục tối đa 5 cấp, đăng ký file bằng hash/size và bỏ qua file ẩn hoặc `.part`. Scanner dùng FOLLOW_LINKS; phạm vi chia sẻ và symlink cần được quyết định rõ nếu muốn giới hạn dữ liệu trong một thư mục thực. File meta kết thúc `.landrop-meta` chưa được filter theo hậu tố trong cùng logic.

Các chức năng bổ sung có trong backend:

- `FolderArchiveService`: tạo ZIP tạm từ thư mục, truyền ZIP bằng engine file và hỗ trợ giải nén; có kiểm tra normalize/startsWith để chặn ZIP entry vượt thư mục đích. Chưa thấy quota dung lượng/số entry khi giải nén.
- `ClipboardMessage` và `ChatService`: gửi text clipboard theo hành động rõ ràng của phía gọi; không có vòng tự đồng bộ clipboard ngầm trong service này.
- `ChatHistory`: lịch sử chat peer lưu trong RAM, khác với lịch sử channel lưu PostgreSQL.

10. Tài khoản, phiên và thiết bị

10.1. Đăng ký và đăng nhập

Register kiểm tra email, username 3-30 ký tự theo pattern và password tối thiểu 8 ký tự, chuẩn hóa email, từ chối email/username trùng và tạo user. Password được băm bằng **PBKDF2-HMAC-SHA256, 600.000 vòng**, salt ngẫu nhiên 16 byte và khóa 256 bit; verify dùng so sánh constant-time của MessageDigest.

Login nhận identifier/email/username, kiểm tra password, trạng thái user và ranh giới user/admin. Tài khoản SYSTEM_ADMIN phải dùng route đăng nhập Control Center. DTO user trả ra không chứa password hash.

JWT dùng HS256, có subject user ID, role, type, iat, exp, jti và sid. Refresh token được lưu **hash SHA-256**, không lưu token thô, kèm user ID, thời hạn, revoked và device ID. Refresh thu hồi token cũ rồi cấp cặp token cùng session mới; đây là rotation. Nếu bước cấp mới thất bại sau khi thu hồi, hai thao tác chưa được bao bọc trong một transaction chung.

10.2. Các định danh khác nhau

Định danh	Mục đích	Vòng đời
userId	Tài khoản	Lâu dài
sessionId	Phiên đăng nhập/refresh	Đổi khi phát hành phiên mới
clientInstanceId	Bản cài đặt client	Ổn định theo phía client
deviceId	Thiết bị đã enroll	Server cấp và lưu DB
connectionId	Một socket realtime	Đổi khi reconnect
peerId/endpoint	Địa chỉ logic/vật lý trong LAN	Không thay thế danh tính tài khoản

`SessionVerifier` kiểm tra session tồn tại, thuộc đúng user, chưa revoked và chưa hết hạn. Foundation HTTP middleware được truyền đầy đủ user/session repository, nên kiểm tra cả token và trạng thái hiện tại trong DB. Logout thu hồi đúng phiên và đóng kết nối được bind với phiên đó; các phiên khác của cùng user có thể tiếp tục.

10.3. Enroll thiết bị và online

`DeviceService` tìm clientInstanceId, giữ device ID nếu cùng chủ sở hữu, báo conflict nếu bản cài đặt thuộc user khác, upsert metadata rồi bind session vào device. API rename kiểm tra ownership. Online trong Foundation lấy từ registry realtime đang mở; last-seen DB chỉ là lịch sử.

Ở bộ file/transfer cũ có DTO thiết bị khác và tiêu chí “last seen trong 600 giây” để coi online. Hai nghĩa online này khác nhau. Có `DeviceHttpHandler` và route thiết bị của `FileHttpServer`; Foundation ưu tiên handler enroll mới, còn AuthHttpServer hiện gắn route cũ và tạo DeviceHttpHandler nhưng chưa register nó.

10.4. Khoảng trống bảo mật có căn cứ

JWT secret có fallback cố định trong AuthConfig; kiểm tra production ở bootstrap mới bắt buộc mật khẩu admin, chưa bắt buộc JWT secret riêng. Không thấy rate limiter trên login/register trong các handler được đọc. HTTP/WebSocket mặc định mở qua HTTP/WS, chưa thấy HttpsServer/TLS factory được cấu hình ở entrypoint này.

Đây là các điểm cần khắc phục ở deployment hoặc code trước khi đưa server ra môi trường không kiểm soát. Báo cáo không kết luận một hệ thống khác đã bổ sung proxy/TLS ở ngoài repo.

11. Workspace, phân quyền và chat bền vững

Workspace là không gian nhóm gồm owner, thành viên, channel, folder và file. `WorkspaceService` tạo workspace và membership OWNER trong cùng transaction. Membership và quyền được kiểm tra qua `WorkspaceAccess`; một số thao tác lock workspace bằng `FOR UPDATE` để tránh thay đổi cạnh tranh.

Hành động	OWNER	ADMIN	MEMBER
Xem workspace, thành viên và nội dung được phép	Có	Có	Có
Sửa thông tin workspace	Có	Có	Không
Thêm member bằng email	Có	Có	Không
Đổi role ADMIN/MEMBER	Có	Không	Không
Xóa workspace	Có	Không	Không
Rời workspace	Bị chặn	Có	Có
Tạo channel	Có	Có	Không
Sửa/xóa message của người khác	Không có quyền này qua chat service hiện tại	Không	Không

Owner được bảo vệ khỏi bị hạ quyền hoặc xóa membership. Thông báo leave gợi ý transfer ownership, nhưng chưa thấy API chuyển owner được triển khai trong WorkspaceService này. GUEST có trong enum/schema, song nhiều API chủ động không hỗ trợ quyền truy cập của GUEST.

`ChannelChatService` quản lý channel và tin nhắn bằng SQL, có history cursor theo cặp thời gian + UUID. Keyset này giúp phân trang ổn định khi nhiều message có cùng timestamp. Message tối đa 4.000 ký tự; reply phải cùng channel; edit/delete chỉ cho sender còn quyền truy cập. Xóa message là soft delete, thay nội dung bằng placeholder.

`clientMessageId` và unique index `(user_id, client_message_id)` chống tạo trùng khi gửi lại sau mất ACK. Nếu cùng ID được dùng cho nội dung/channel khác, service báo conflict. Ghi DB được commit trước khi broadcast, nên realtime không được dùng làm nguồn dữ liệu duy nhất; lịch sử DB giúp client khôi phục khi reconnect.

Có điểm policy chưa thống nhất: `JdbcRealtimeStore.canAccess()` kiểm tra `is_private = FALSE`, nhưng `ChannelChatService.requireChannelAccess()` chủ yếu kiểm tra workspace membership và chưa kiểm tra cơ chế riêng của private channel. API create channel hiện tạo channel public. Vì vậy không nên coi private channel là chức năng phân quyền đã hoàn chỉnh chỉ vì schema có cột `is_private`.

12. Hai bộ WebSocket và sự khác biệt khi tích hợp

Tiêu chí	FoundationRealtimeServer	RealtimeServer
Entry point	LandropServerMain / ServerBootstrap	AuthHttpServer.main
Handshake	Event hello, trả welcome, protocol version	Auth tại HTTP upgrade, trả connected
Identity	user/session/device/clientInstance /connection	user/token ID/device label/socket
Registry	ClientRegistry	ConnectionRegistry
Chat	Join/leave và nhận broadcast từ REST chat service	Send/edit/delete/history và room trực tiếp trên WS
Transfer	Điều phối in-memory theo connection ID	Service lưu DB theo user/file/device
Session revoke	Có bind và đóng socket theo session	Authenticator mặc định kiểm tra token và user tồn tại; chưa kiểm tra session registry
Kiểm tra thời hạn	Session/account ở heartbeat của socket đã auth	Access token expiry, sweep định kỳ
Hợp đồng event	Envelope gồm payload và request ID	Commands/events phẳng; có alias dấu hai chấm

Đây là khác biệt về contract, không chỉ tên lớp. Client tích hợp phải chọn đúng server và đúng dạng message. RealtimeServer cũ còn từ chối upgrade có header Origin, một dấu hiệu contract được thiết kế thiên về native client; không thể giả định web client mới dùng trực tiếp mà không kiểm tra đường kết nối.

Wiring chat Foundation: REST chat gọi ChannelChatService, service broadcast cho FoundationRealtimeServer và server gửi tới subscriber còn membership. **Wiring transfer Foundation:** service transfer DB được tạo nhưng chưa được set broadcaster tương đương đường AuthHttpServer.main; transfer WS Foundation dùng TransferCoordinator in-memory khác. Như vậy các event transfer DB và thống kê transfer in-memory không tự động cùng một nguồn dữ liệu.

Khoảng trống cần xử lý ưu tiên: khi có đầy đủ auth dependency, Foundation vẫn có thể welcome một hello không token/device. `validateRuntimeSession()` trả true cho connection chưa authenticated; handler transfer không có gate xác thực chung. Do đó đường điều phối transfer sau hello có thể được gọi trước khi bind danh tính, nếu biết receiver connection ID. Tài liệu Foundation-04A mô tả các action đó bị chặn, nhưng code snapshot chưa bảo đảm đúng mô tả. Nhận xét này dựa trên đọc control flow, chưa có exploit integration chạy trong lần phân tích này.

13. Metadata file/folder và transfer có DB

`FileManagementService` quản lý metadata của file và cây folder theo workspace. Create derive uploader từ user đã auth; update/delete cho uploader hoặc OWNER/ADMIN; folder có quyền creator tương tự. Move folder kiểm tra workspace, trạng thái deleted và vòng lặp cha-con bằng recursive query. File/folder dùng soft delete với `deleted_at`.

`storageType` có LAN_ONLY, LOCAL, SERVER, OBJECT_STORAGE. Nhưng create metadata nhận tên/size/hash/type và lưu record; chưa phải endpoint upload byte file. “File đã đăng ký” không đồng nghĩa “server đã lưu bản file” hoặc “một peer đang online giữ file”. Xóa metadata cũng không tự gửi lệnh xóa bản file trên máy peer.

`TransferCoordinationService` yêu cầu cả sender và receiver có membership trong workspace của file, lưu transfer vào DB rồi emit cho user/device khi có broadcaster. Các action gồm request, accept, reject, cancel, progress, complete và fail. State machine đi từ PENDING sang ACCEPTED, TRANSFERRING và terminal. API complete ghi nhận báo cáo của participant; server điều phối không tự đọc file đích để xác nhận hash.

Có hai nhóm TransferRecord/TransferStatus: nhóm `database.model`/`file.model` lưu DB và nhóm `server.transfers` lưu RAM. Chúng có ID, quyền và vòng đời khác nhau. Cần chuẩn hóa điểm nối giữa “request được phép”, “transport đang gửi byte” và “kết quả đã được xác minh”, tránh đánh dấu hoàn tất một lớp trong khi lớp khác chưa hoàn thành.

14. Mô hình dữ liệu và migration

PostgreSQL chứa **10 bảng nghiệp vụ chính**. ID phần lớn là UUID; devices dùng chuỗi. Repository dùng PreparedStatement, RETURNING để lấy row sau insert/update và try-with-resources cho connection/statement/resultset. Pool độc lập theo schema có hỗ trợ để phục vụ một số kiểm thử cô lập.

Bảng	Vai trò và quan hệ
users	Tài khoản, password hash, status và system role
refresh_tokens	Phiên, token hash, expiry, revoke, device ID
devices	Bản cài đặt của user, thông tin platform, endpoint và last seen
workspaces	Nhóm, slug và owner
workspace_members	User tham gia nhóm, role; unique workspace + user
channels	Kênh thuộc workspace, tên/topic/is_private
messages	Sender, channel, reply, clientMessageId, edited/deleted timestamps
folders	Cây thư mục, creator và soft delete
files	Metadata, uploader, folder, size/hash và storage fields
transfers	File, sender/receiver user/device, trạng thái và timestamps

Migration V1 tạo workspace/chat/file nền tảng; V2 thêm auth session; V3 thêm message identity; V4 thêm edited/deleted message; V5 thêm file/folder fields, devices và transfers; V6 thêm status/system role; V7 bổ sung enrollment và session/device binding.

`DatabaseMigrator` áp dụng danh sách V1-V7 tuần tự, transaction riêng cho mỗi script. Không thấy bảng migration history/checksum; cơ chế phụ thuộc nhiều vào SQL IF NOT EXISTS và script được chạy lại. `verifyTablesCreated()` kiểm tra schema `public`; phần log/comment vẫn ghi V1-V4 dù danh sách đã tới V7.

Startup Foundation chưa gọi migrate. Bootstrap hiện kiểm tra DB bằng `SELECT 1`, sau đó tạo service/repository. `DatabaseCli migrate` là đường áp dụng migration được thấy trong code. Do đó cần chuẩn bị schema trước khi chạy server trên database mới. Nếu tài liệu hoặc quy trình triển khai giả định startup tự migrate thì cần điều chỉnh giả định đó.

Có chỗ Prisma mô tả relation refresh token -> device với onDelete SetNull, trong khi migration SQL V7 chỉ thêm cột device_id/index, chưa thêm foreign key tương ứng. Schema mô tả và schema được SQL tạo cần được đối chiếu trước khi coi hoàn toàn đồng nhất.

15. Control Center và quan sát vận hành

Trang `admin/index.html` là giao diện HTML/CSS/JavaScript có sẵn trong backend, được trả tại `/admin`. Nó gọi các API quản trị, hiển thị dashboard, user, workspace/member, session, client, interface mạng, cấu hình và log. Token admin được lưu ở sessionStorage và logout gọi route thu hồi phiên.

`AdminControlCenterService` có hành vi thực tế: disable user rồi revoke session/disconnect socket, enable user, quản lý display name/system role, chặn tự xóa/tự hạ quyền, bảo vệ admin cuối cùng khi xóa/hạ quyền, quản lý workspace/membership và disconnect client. Dashboard có uptime, user/session/device/workspace counts cùng active transfer in-memory.

`OperationalLogService` là ring buffer **1.000 entry trong RAM**, có filter và giới hạn page. Log này mất khi process restart; chưa phải audit trail bền vững. `AppLogger`, `ServerEventLogger` và log của thư viện tồn tại song song, với một số pattern che password/token/secret. Regex masking không thay thế việc tránh log dữ liệu nhạy cảm ở nguồn.

`AdminBootstrapService` có default password dành cho phát triển, hỗ trợ password từ environment/system property và option tạo random. Constructor mặc định không preserve password cũ: ensureSystemAdminExists có thể đặt lại password và status/role của account cố định mỗi lần khởi động. Production chỉ kiểm tra mật khẩu cấu hình tối thiểu 12 ký tự và một số placeholder. Cần chọn policy bootstrap một lần/rotation rõ ràng nếu muốn giữ thay đổi admin qua restart.

Health/status công khai hữu ích để discovery/diagnostics; status cũng trả host/port/LAN addresses/connected clients. Đây là thông tin vận hành nên được cân nhắc phạm vi công khai khi triển khai. Không thấy metric exporter, tracing hay log sink tập trung được cấu hình trong module.

16. Bản đồ API để tích hợp

Các route sau là nhóm hợp đồng tiêu biểu, không thay thế tài liệu schema đầy đủ. Prefix chung thường là `/api`; HTTP method và payload phải đối chiếu đúng handler của entrypoint đang dùng.

Nhóm	Route tiêu biểu	Công việc
Health	GET /health, /api/health; GET /status, /api/status	Health/uptime và thông tin server Foundation
Auth	POST /api/auth/register, /login, /refresh, /logout; GET /api/auth/me	Tài khoản và phiên
Device mới	POST /api/devices/enroll; GET /api/devices; GET/PATCH /api/devices/{id}	Enrollment, danh sách và rename
Workspace	GET/POST /api/workspaces; GET/PATCH/DELETE /api/workspaces/{id}	Nhóm
Member	GET/POST .../{id}/members; PATCH/DELETE .../{id}/members/{userId}; POST .../{id}/leave	Thành viên và role
Channel	GET/POST /api/workspaces/{id}/channels; GET /api/channels/{id}	Kênh chat
Message	GET/POST /api/channels/{id}/messages; PATCH/DELETE /api/messages/{id}	History, gửi, sửa và xóa
File/folder	GET/POST /api/workspaces/{id}/files hoặc /folders; GET/PATCH/DELETE /api/files/{id} hoặc /folders/{id}	Metadata và cây thư mục
Transfer DB	POST /api/transfers; GET /api/transfers/{id}; POST .../{id}/{action}	Điều phối transfer theo user/file
Admin auth	POST /api/admin/auth/login	Đăng nhập SYSTEM_ADMIN
Admin dữ liệu	/api/admin/users, /workspaces, /sessions, /clients, /logs, /dashboard	Quản trị
Server info	/api/server/info, /interfaces, /config	Vận hành và cấu hình

Một số handler trả JSON trực tiếp, một số trả `{data: ...}`, lỗi auth dạng flat, lỗi workspace/chat/file dạng `{error: {code, message}}`. Giới hạn body cũng khác: workspace 16 KiB, chat 32 KiB, auth/device mới đọc tối đa 64 KiB, trong khi một số handler cũ/file dùng readAllBytes. Client cần adapter hoặc chuẩn hóa contract nếu muốn xử lý lỗi/phân trang thống nhất.

17. Kiểm thử thực tế và độ tin cậy của bằng chứng

Đã chạy tại Windows 11 với **Maven 3.9.16, Oracle Java 26.0.2.1**, compiler target/release của dự án là 21:

```
mvn -pl backend "-Dlandrop.database.enabled=false" "-Dlandrop.test.postgres=false"
test
```

Hai property được truyền để không dùng database cấu hình hiện có và không bật các PostgreSQL suite opt-in. Kết quả Maven **BUILD FAILURE**; không chỉnh code để làm thay đổi kết quả phân tích.

Chỉ số Surefire	Kết quả
Tests run được Maven tổng kết	291
Thành công trong số được tổng kết	204
Assertion failures	0
Errors	63
Skipped được Maven tổng kết	24
Thời gian Maven báo	11,981 giây

Các lỗi tập trung ở HTTP/WebSocket integration. Một nhóm lỗi xuất phát từ `HttpClient.newHttpClient()` không khởi tạo được selector/pipe loopback; stack trace có `UnixDomainSockets.connect0` và `SocketException: Invalid argument: connect`. Nhóm realtime ghi `Realtime listener failed`. Những lỗi này xảy ra ở setup/listener, nên các kịch bản tương ứng chưa được kiểm chứng. Không có cơ sở coi toàn bộ 63 lỗi là bug nghiệp vụ; cũng chưa đủ bằng chứng quy hết cho Java 26.

24 skipped trong lần chạy này là các case của WorkspacePostgres, RealtimePostgres, PersistentChatPostgres và FileManagementPostgres. Ngoài ra, AuthIntegration, DatabaseIntegration và Foundation04AuthenticationDevice có assumption ở BeforeAll khi không có DATABASE_URL; log của chúng ghi tests=0. Vì vậy, số “24 skipped” không phải toàn bộ số kịch bản DB chưa chạy.

Các suite protocol, crypto, nhiều luồng peer/discovery và transfer đã hoàn thành thành công ở lần này. Bộ test có ý định bao phủ corrupted packet, collision tên file, hash mismatch, resume sai metadata/file nguồn thay đổi, failover, late peer join, chat trong lúc tải, ownership thiết bị, revoke session và phân quyền workspace/chat/file.

Giới hạn file lớn: `LargeFilePeerToPeerIntegrationTest` dùng file 512 KiB và 1 MiB; benchmark dùng 2 MiB và 1 MiB. Chúng kiểm chứng cơ chế nhiều chunk ở quy mô nhỏ, không chứng minh truyền 1-5 GiB hoặc giới hạn 5 TB mà lớp điều phối cho phép khai báo. Benchmark loopback cũng không tương đương tốc độ mạng Wi-Fi/LAN thật.

Kết quả chi tiết được lưu cùng báo cáo trong CSV trích từ log riêng. XML Surefire đã được một lần chạy khác ghi đè trong lúc lập báo cáo nên không dùng để thay đổi kết luận của lần chạy này. Bước kiểm chứng tiếp theo là chạy lại trên JDK 21 đúng baseline, xử lý điều kiện network/listener, rồi bật PostgreSQL suite với database chuyên dùng để kiểm thử. Các test mới nên nhắm vào khoảng trống có hành vi thực tế.

18. Các phát hiện và thứ tự ưu tiên

Mức “cao” nghĩa nên xử lý trước khi tuyên bố chức năng tương ứng hoàn thiện hoặc mở server ra môi trường không kiểm soát. Đây là ưu tiên kỹ thuật từ phân tích tĩnh, không phải chấm điểm pentest hoặc CVSS.

ID	Ưu tiên	Phát hiện	Tác động và hướng xử lý
F01	Cao	Chunk 2 MiB vượt giới hạn payload 2 MiB khi cộng metadata/mã hóa	File >2 GiB dùng nhánh này có thể bị decoder từ chối. Đồng bộ giới hạn và thêm test ranh giới.
F02	Cao	Foundation thiếu gate xác thực chung trước transfer action	Hello chưa bind identity vẫn có đường điều phối transfer. Chặn action đến khi auth/session/device hợp lệ.
F03	Cao	Hai entrypoint có auth/session/device policy khác nhau	Cùng route/port mặc định nhưng bảo đảm revoke và ownership khác. Chọn entrypoint chuẩn và dùng chung policy.
F04	Cao	JWT secret fallback cố định, HTTP/WS chưa cấu hình TLS tại server	Bắt buộc secret phù hợp cho môi trường triển khai và xác định đường HTTPS/WSS.
F05	Cao	ECDH P2P chưa có chứng thực peer; enforcement chưa chặn plaintext nhận	Hoàn thiện binding danh tính/khóa và kiểm tra cả hai chiều, tách READY khỏi SECURE khi gọi engine.
F06	Cao	Foundation không tự migrate và health chưa chứng minh schema sẵn sàng	Database mới có thể thiếu schema dù process đã trả health. Chuẩn hóa migration và readiness theo mode.
F07	Trung bình	Transfer DB broadcaster chưa được nối vào Foundation; dashboard đếm transfer RAM	Event và thống kê không tự bao quát transfer DB. Nối service và xác định nguồn trạng thái chuẩn.
F08	Trung bình	File cấu hình current/pending chưa trở thành cấu hình hiệu lực sau restart	UI lưu cấu hình nhưng process có thể vẫn đọc environment cũ. Nối load/apply và phản ánh đúng restart requirement.
F09	Trung bình	Resume chưa lưu đủ context bền vững; shutdown có thể xóa partial	Giới hạn khả năng khôi phục sau restart. Thiết kế catalog transfer nếu đây là yêu cầu sản phẩm.
F10	Trung bình	Quyền private channel khác nhau giữa chat service và realtime store	Có nguy cơ hành vi truy cập không nhất quán nếu dữ liệu private được tạo từ đường khác. Hợp nhất policy.
F11	Trung bình	Body limit, error shape, online/device contract khác nhau	Khó tích hợp frontend và khó vận hành nhất quán. Chuẩn hóa transport adapter/contract.
F12	Trung bình	Bootstrap admin đặt lại password/status theo cấu hình mỗi startup	Thay đổi admin qua UI/công cụ có thể không bền. Xác định policy bootstrap và rotation.
F13	Trung bình	Test GB/DB/network thực chưa được xác nhận; lần chạy có 63 errors	Chưa có bằng chứng nghiệm thu toàn hệ thống. Chạy baseline JDK21 và test matrix thực.
F14	Thấp	Tài liệu đang được tổ chức và cập nhật cùng lúc	Kiểm tra lại link, tên entrypoint và giới hạn tính năng sau khi chốt thay đổi; không coi mô tả README cũ là lỗi còn tồn tại.

Một số điểm bổ sung nên kiểm tra khi làm hardening: state update transfer RAM chưa là compare-and-set transaction; transfer map/log/history có vòng đời dữ liệu chưa đồng nhất; shared scanner theo symlink; extract ZIP chưa có quota; single-peer cho phép offer thiếu hash; validation metadata và partial write cần xử lý chặt ở ranh giới. Các nhận xét này không được dùng để suy luận đã có mất dữ liệu trên máy người dùng.

19. Đề xuất lộ trình hoàn thiện

Bước 1 - Khóa hợp đồng server. Chọn LandropServerMain làm entrypoint chuẩn hoặc ghi rõ lý do dùng entrypoint khác; thống nhất login/session/device policy, WS event envelope và route device. Gắn các service/broadcaster còn thiếu. Kết quả cần đạt là một client dùng một contract có thể login, enroll, join channel và tạo transfer với danh tính đã kiểm tra.

Bước 2 - Sửa các giới hạn có thể làm hỏng tính năng. Đồng bộ max packet/chunk/encryption overhead; kiểm tra trạng thái transfer chuyển cạnh tranh; đảm bảo request/response correlation và zero-byte/hash policy rõ ràng. Test mới tập trung >2 GiB boundary, plaintext nhận khi enforce, action trước auth và duplicate transition.

Bước 3 - Hoàn thiện vận hành và dữ liệu. Áp dụng migration có kiểm soát trước readiness; thống nhất schema SQL/Prisma; kiểm chứng build Docker hiện đã dùng backend và dependency runtime; nối cấu hình lưu file với startup; quyết định log persistence và session cleanup. Chỉ đánh dấu ready khi các dependency của mode đang chọn thực sự sẵn sàng.

Bước 4 - Nghiệm thu sản phẩm. Chạy JDK21/Maven baseline, PostgreSQL riêng, hai máy LAN, file 1/3/5 GiB, mất kết nối/mất nguồn/thay đổi file nguồn, reconnect và đa thiết bị. Đo tốc độ, CPU, RAM, disk và thời gian verify; phân biệt kết quả loopback với LAN thật. Nếu cần Internet/NAT/cloud storage, coi đó là yêu cầu phát triển riêng vì engine hiện tại chưa tự cung cấp.

20. Kết luận về dự án

LANDrop đã phát triển từ công cụ truyền file/chat trực tiếp trong LAN thành nền tảng có tài khoản, nhóm, lịch sử và quản trị. Backend có nhiều thành phần kỹ thuật đáng kể: framing TCP rõ ràng, xử lý chunk, bitmap resume, positional write, tải đa nguồn, kiểm tra toàn vẹn, password hashing và session-device binding.

Tôi xem **engine LAN là nền tảng kỹ thuật chính**, **workspace/chat/database là tầng cộng tác**, và **Foundation/Admin là tầng vận hành**. Ba tầng đã có code, nhưng một số đường nối và policy chưa đồng nhất. Phần việc có giá trị nhất tiếp theo là hoàn thiện sự nhất quán đó và kiểm chứng các ranh giới lớn, thay vì chỉ tăng thêm số tính năng.

Kết luận này dựa trên mã nguồn tại snapshot và kết quả kiểm thử đã nêu, không dựa trên nhãn phase hoặc các cam kết hiệu năng chưa được chạy lại.

Phụ lục A. Căn cứ mã nguồn chính

Các đường dẫn tương đối tính từ root repository. Phụ lục danh mục đi kèm chứa toàn bộ 258 tệp để tra cứu thêm.

Mã	Tệp / vị trí	Căn cứ
S01	backend/pom.xml	Dependency, Java release, main class manifest
S02	server/bootstrap/ServerBootstrap.java:96	Vòng đời startup, wiring DB/HTTP/WS, production password
S03	server/realtime/FoundationRealtimeServer.java:175	Dispatch heartbeat/action; auth gate
S04	server/realtime/FoundationRealtimeServer.java:317	HELLO, đăng ký connection, welcome/auth binding
S05	server/realtime/FoundationRealtimeServer.java:737	authRequired và runtime session validation
S06	auth/server/AuthHttpServer.java:68	Constructor/wiring route cũ, admin/device dependency
S07	auth/server/AuthHttpServer.java:392	Entry point Auth và nối broadcaster realtime
S08	auth/server/AuthMiddleware.java:48	Kiểm tra token, session và user nếu repository được cung cấp
S09	auth/security/PasswordHasher.java:19	Thuật toán PBKDF2, salt và iterations
S10	auth/security/SessionVerifier.java:16	Session ownership, revoke và expiry
S11	auth/config/AuthConfig.java:24	Fallback JWT secret
S12	device/service/DeviceService.java:55	Enrollment, ownership và bind session/device
S13	protocol/PacketEncoder.java:31	Header version/type/length/requestId
S14	protocol/PacketDecoder.java:19	Payload maximum và decode validation
S15	transfer/FileChunk.java:121	Chọn chunk size theo file size
S16	transfer/messages/ChunkResponseMessage.java:48	Binary payload cộng metadata
S17	peer/PeerConnection.java:222	Dispatch và nhận plaintext/encrypted packet
S18	peer/PeerConnection.java:372	Đường gửi mã hóa/enforcement
S19	security/P2pCryptoSession.java:64	ECDH, AES-GCM và fingerprint
S20	transfer/FileTransferService.java:321	Offer, accept, chunk pull, resume và finalize
S21	transfer/ChunkManager.java:316	Lưu/load bitmap meta
S22	transfer/MultiPeerDownloadCoordinator.java:161	Worker, schedule và verification
S23	transfer/SharedFileScanner.java:47	Depth, symlink và filter file chia sẻ
S24	chat/service/ChannelChatService.java:101	Channel access, history, idempotent save, edit/delete
S25	realtime/JdbcRealtimeStore.java:16	Điều kiện truy cập và lưu chat
S26	realtime/SocketAuthenticator.java:10	Auth của bộ realtime cũ
S27	file/service/TransferCoordinationService.java:103	Transfer lưu DB và emit

Mã	Tệp / vị trí	Căn cứ
S28	database/service/WorkspaceService.java:24	Transaction và quyền workspace/member
S29	database/migration/DatabaseMigrator.java:47	Migration V1-V7 và verify public schema
S30	server/config/ServerConfigManager.java:55	Save/load pending, requiresRestart
S31	admin/security/AdminBootstrapService.java:129	Bootstrap admin và reset/preserve password
S32	admin/logging/OperationalLogService.java:16	Ring buffer log
S33	src/main/resources/admin/index.html	Giao diện Control Center
S34	src/test/java/com/landrop/transfer/LargeFilePeerToPeerIntegrationTest.java:78	File kiểm thử chỉ 512 KiB và 1 MiB
S35	src/test/java/com/landrop/benchmark/PerformanceBaselineTest.java:122	Benchmark file 2 MiB

Đối với S02-S32, package root đầy đủ là `backend/src/main/java/com/landrop/`. Đối với S33-S35, root là `backend/`. Line locator dùng để định hướng theo snapshot; tên phương thức và nội dung code là căn cứ chính khi file thay đổi.